Memory Card Game – Project Write-Up

Overview

The Tkinter graphical user interface was used to construct this simple memory card game (concentration/matching pairs) in Python. The deck is shuffled, unique card labels (words) are read from a text file (cards.txt), duplicates are made to form pairs, and the results are shown in a 4x4 grid. After clicking cards to see the text, the player tries to match pairings.

A memory game is a single-file project that, while still being doable within the project's time limits, can illustrate numerous programming concepts needed for the course.

How the Project Meets the Course Requirements

- Loops: `for` loops are used to create widgets, iterate files, and process data.
- Variables: strings, numbers, lists, and tuples are used throughout.
- Functions: functionality is separated into methods and functions
- Processing Data: reads card words from *cards.txt* and writes scores to *scores.csv*.
- Graphics: GUI implemented with *Tkinter* (buttons represent cards).
- Exception Handling: file I/O and parsing are surrounded by try/except blocks with user-friendly messages.
- Classes: includes `Card` and `MemoryGame` classes.
- GUI: uses *Tkinter* for the user interface.

Files Included

- `memory_game.py` – Main program
- `cards.txt` – Sample card words (one per line)
- `scores.csv` – Scores file (CSV), initially with header
- `README.txt` – Project documentation
- `video_script.txt` – Short script for a 3-minute demonstration
- `flowchart.png` – Simple flowchart of the program logic

Pseudocode / Algorithm

1. Read unique words from *cards.txt* (one per line).
2. Verify there are enough unique words to fill half the board (pairs needed).
3. Duplicate the list so every word appears twice.
4. Shuffle the combined list.
5. Create a grid of `Card` objects (buttons) using *Tkinter*.
6. On user click:
    a. If it is the first card, reveal it.
    b. If it is the second card, reveal it and check match:
        i. If match: mark both as matched and disable them.
        ii. Else: hide both after a short delay.
    c. Increment move counter and update the timer display.
7. When all pairs are matched, prompt the user to save score and show win message.
8. Scores are appended to *scores.csv* in CSV format.

Possible Enhancements

- Support variable board sizes selected by the user (e.g., 2×4, 4×4, 6×6).
- Add images instead of words (load image files listed in *cards.txt*).
- Add sound effects on match/miss.
- Improve styling and animations.

Challenges Anticipated

- Ensuring images scale correctly if images are used.
- Handling various sizes of the cards file (too few or too many entries).
- Cross-platform differences in *Tkinter* fonts and appearance.